Are LLM Written Unit Tests Flaky?

Progress Report 1, ECE 382V Software Testing in the Era of Nondeterminism, Fall 2026

Shaunak Kapur (sk62288)
The University of Texas at Austin
Austin, Texas, USA

Manas Pathak (map7848)
The University of Texas at Austin
Austin, Texas, USA

1 SUMMARY

We finished everything we planned for this milestone and moved two items from the second milestone forward. We now have 72 LLM written test suites for 12 Python libraries, made with three models or setups and three prompt styles, plus the 12 human written suites as the baseline. Every suite has been rerun under random test order and hash seed, ten time zones, and two kinds of machine load, and every flaky test has a labeled root cause. After the proposal feedback we changed the method in five places. Subjects now come from a written selection funnel, two new prompt styles are based on prior work and on how developers actually ask for tests, generated tests are also run inside the human suites, machine load now follows Shaker [6], and every scanner rule is tied to a root cause category and checked against flaky tests labeled by Gruber et al. [1].

Counting every rerun, mixed suite, and targeted run, the study has now executed about 2.6 million tests. The main results so far are these. Flaky LLM tests are rare and all of them come from one setup (one shot Haiku with the feature prompt). The two new prompt styles produced no flaky tests in 2,104 tests and about a sixth of the risky patterns of the feature prompt, which means our original prompt was a large part of why LLM tests looked risky. The new mixed suites found something the separate runs could not. Three agentic tests for schedule pass every time in their own suite but fail in up to 22 of 30 runs once they share a session with the human tests, because the human tests leave jobs in the library's global scheduler. Stress-ng found one new flaky human test.

2 GOALS FOR THIS MILESTONE

Table 1 lists the goals from our proposal and where each one stands.

We met every goal. The one thing that did not go as hoped is the number of flaky tests. Flakiness is rare in all suites, so the confidence intervals for RQ1 overlap and the comparison between human and LLM tests is weak if it only counts flips. This is why we added the risk measures, the mixed suites, and the stronger machine load described below, and why the mitigation will not be measured only on observed flips.

Table 1: Proposal goals for Intermediate Steps 1 (by October 23)

Goal	Status
Finish the subject list	Done. Now backed by a written funnel (Section 3.1)
Generate agentic and both one shot suites	Done October 1 (36 suites). 24 more added October 22 with two new prompt styles
Full rerun experiment, all three changes	Done for all 84 suites, about 1.1 million test executions
RQ1 flaky rates	Done, Table 2
RQ2 root cause of every flaky test	Done, Section 4.2
First version of the static scanner	Done, and validated on outside labels (Section 3.5)
Moved forward from Intermediate Steps 2	
1,000 reruns of flagged tests on a slow Linux runner	Done October 4 (303,420 executions) and again with stress-ng (164,420 more executions, October 22)
Plain prompt that does not name time features	Done October 22

3 WHAT WE CHANGED AFTER THE PROPOSAL FEEDBACK

3.1 How subjects are chosen

`scripts/select_subjects.py` applies fixed rules and logs the count after each one. It starts from the 5,000 most downloaded PyPI packages over the last 30 days and keeps packages that are libraries (no typing stubs or pytest plugins), link a GitHub repository, ship a pure Python wheel, have at most three required dependencies and run on Python 3.12, and have test files in their repository. The counts are 5,000, 4,701, 4,131, 3,296, 2,188, and 1,959. The survivors are split into two strata by whether the library source imports a module tied to nondeterminism (time, datetime, random, threading, asyncio, concurrent, multiprocessing, tempfile, socket). 1,135 do and 824 do not. We oversample the first stratum on purpose, since that is where flaky tests come from, and we say so. The last rule is that the suite passes offline (pytest-socket) in under 60 seconds at a pinned commit. It needs a clone and a run, so it is applied to a seeded random sample. In a sample of 6 per stratum, 5 of 12 passed. These 5 are the pool we will draw new subjects from. All 12 of our current subjects pass every rule. 10 are in the nondeterminism stratum and 2 (tinydb and tabulate) are not. Their download ranks run from 44 (filelock) to 2,167 (tinydb). The same script can be rerun with a different seed to add subjects in the same way.

3.2 Which prompts, and why

Our original prompt names the time, random, thread, and file features of each library so that the model tests them. That makes risky tests more likely, so we now treat it as an upper bound and call it the *feature prompt*. We added two styles, both run one shot with Claude Sonnet so they compare directly with the feature prompt suites (OS-S). **TP-S** follows TestPilot [5]. The model sees only the signatures, docstrings, and README usage examples of the public API (built by `scripts/build_testpilot_context.py`), never

the implementation. **PL-S** is a plain developer prompt, "write unit tests for this library using pytest", with the source files and no hints about what to test. All styles keep the same practical limits (50 to 80 tests, pytest only, no reading of the existing tests, no running). The exact prompt texts are in `prompts/README.md`.

3.3 Running generated tests with the existing ones

So far each generated suite ran as a whole in random order, which can expose order dependence inside that suite, and each flaky test was then rerun alone only to diagnose it. We now also run a *mixed* condition (`scripts/mixed.py`). The generated tests a pipeline would keep are copied into the library's own test folder and pytest runs both in one session in random order, which is how a developer would add them. Either side can now pollute the other. The human suite is run alone on the same machine as the baseline.

3.4 Machine load

The original stress mode starts one Python busy loop per core, which only loads the CPU and was not based on prior work. We added a `stressng` mode that follows Shaker [6]. It uses the same four stress-ng options as Shaker (`--cpu`, `--cpu-load`, `--vm`, `--vm-bytes`) and Shaker's first step, a uniform random sample of ten configurations over those domains. Run i uses configuration i mod 10 and the configuration of every run is saved, so once we have enough flaky tests we can build Shaker's configuration by test matrix and pick the configurations that expose the most. The limits are that we load only CPU and memory, not disk, network, or garbage collection, and that we use two machine types (an 8 core Apple M3 and 2 vCPU Linux runners).

3.5 The static scanner

Each rule now names the root cause category it is meant to catch. Real sleeps map to async wait and time [3], unfrozen clock reads and elapsed time asserts to time, unseeded randomness to randomness [1], threads and async tasks to concurrency, fixed file paths and subprocesses to IO and resource leaks, and asserting the order of a set or dict view to unordered collections [2, 3]. We checked the scanner against outside labels. Gruber et al. published a hand labeled sample of 100 non order dependent flaky tests from 100 PyPI projects. We checked out 96 of them at the recorded commit (`scripts/validate_scanner.py`). The scanner flagged 10 of 96. It caught all 3 async wait tests and some of the time and concurrency tests, but only 2 of 35 randomness tests and 1 of 41 network tests. In most misses the nondeterminism is inside the code under test and not in the test body, for example a test that trains a model that seeds itself. Of 714 other tests in the same files that were never reported flaky, 35 were flagged (4.9%). So the scanner is a fair warning for timing problems written directly in the test, which is what the LLM tests have, and it is blind to nondeterminism that lives in the library. A network rule we added after this check raises recall to 13 of 96.

3.6 The human baseline

Human tests are flaky, but rarely. 7 of 5,589 flipped, 4 order dependent (global registries in filelock and arrow), 2 thread races in pybreaker that only show under CPU stress, and 1 tenacity async test that asserts a short sleep finishes quickly and failed once under stress-ng. Since flip counts are small everywhere we now also compare the failure rate of each flaky test, the share of tests with risky patterns, and the results of the mixed suites, not just how many tests flipped.

3.7 If generated tests are never flaky

They are sometimes. Three one shot Haiku tests flip, and both of the tests from our pilot were LLM written. The mitigation step (Intermediate Steps 3) would still have very little to count if it only used observed flips, so it will also be measured on the scanner risk rate, on failures we can trigger on purpose with stress-ng and FlakeRake style delays [4], and on line coverage. We have already drafted the mitigation prompt (`prompts/mitigation_prompt.md`) so that it does not depend on the final list of flaky tests.

4 PRELIMINARY RESULTS

4.1 RQ1, how often

Table 2 has the numbers. Every interval overlaps, so we still cannot say LLM tests are more or less flaky than human tests by flip counts. The two new prompt styles add 2,104 tests with no flips. All three flaky LLM tests are one shot Haiku tests with the feature prompt. Four human tests passed on one machine and always failed on the other (filelock checks that behave differently on macOS, and a toolz version check that fails when the package is installed from a source checkout). We count these as machine dependent and not flaky.

4.2 RQ2, root causes

The causes split cleanly by who wrote the test. All three flaky LLM tests touch real time. One builds an `until` time two hours ahead as a clock time, which is in the past after 22:00, one asserts that two jittered retry waits add up to at least 10 ms (failing 10 to 24% of runs depending on the machine), and one asserts that a retry loop finishes within 120 ms, which failed once under heavy load. The human flaky tests are order dependent (4, an earlier test leaves an object in a class or module level registry), thread races (2), and one timing assertion that only failed under stress-ng.

4.3 Mixed suites

Table 3 shows the generated tests run inside the human suites on GitHub Actions, 30 runs each. No human test became flaky or broke because of the generated tests. The other direction did happen. Three agentic tests for schedule assume the module level default scheduler is empty. The human tests clear it at the start of each test but not at the end, so whenever a human test runs first the generated test sees leftover jobs. Two of these failed in 20 and 22 of 30 runs, and one generated test even ran a job function a human test had registered and crashed with a `TypeError`. These tests never failed in 70 runs of their own suite, so a developer would only see the problem after adding them to the real test suite. One more agentic tenacity test failed in 2 of 30 mixed runs and is still being labeled.

Table 2: Flaky tests by condition. Tests are those a pipeline would keep. A test is flaky if it both passed and failed on the same machine, over 70 laptop runs (random order and hash seed, ten time zones, CPU burners) and 20 stress-ng runs on Linux. Risky is the share of test functions the scanner flags. Coverage is mean line coverage without filelock, whose Windows module never runs.

Condition	Prompt, model, feedback	Tests	Flaky	Flaky % (95% CI)	Risky %	Coverage %	Lost to import errors
Human	written by the developers	5,589	7	0.13 (0.06 to 0.26)	3.9	94.4	0
AG-S	feature prompt, Sonnet agent, runs and fixes	1,511	0	0.00 (0 to 0.25)	8.4	74.1	0
OS-S	feature prompt, Sonnet one shot	1,406	0	0.00 (0 to 0.27)	8.5	81.0	0
OS-H	feature prompt, Haiku one shot	1,595	3	0.19 (0.06 to 0.55)	9.4	67.7	54
TP-S	TestPilot style, Sonnet one shot	913	0	0.00 (0 to 0.42)	1.4	70.7	22
PL-S	plain prompt, Sonnet one shot	1,191	0	0.00 (0 to 0.32)	1.5	82.6	0

Table 3: Generated tests inside the human suites (one pytest session, random order, 30 runs per library, 12 libraries per row)

Mixed in	Gen. tests	Flaky gen.	Human flaky only when mixed	Human tests broken
AG-S	1,511	4	0	0
OS-H	1,597	1	0	0
TP-S	914	0	0	0
PL-S	1,191	0	0	0

4.4 RQ3, risky patterns

The feature prompt suites have about twice the risky pattern rate of human tests (8.4 to 9.4% against 3.9%). The TestPilot style and plain prompt suites from the same model have 1.4% and 1.5%, lower than the human tests. So most of the extra risk came from our prompt pointing the model at time and randomness features, which is what the instructor suspected. The scanner flagged all 3 flaky LLM tests, and its validation on outside labels is in Section 3.5.

4.5 RQ4, the pipeline

The one run filter rejected 101 one shot Haiku tests, 11 TestPilot style tests, and 1 plain prompt test, and none of the rejected tests were flaky. It kept all three flaky Haiku tests, because each one usually passes. One TestPilot style file for schedule did not import, which lost 22 tests. The plain prompt gave the highest coverage of any LLM condition (82.6%), and the TestPilot style prompt, which never sees the implementation, gave 70.7%.

4.6 Machine load and the slow machine

Stress-ng (20 runs per suite, ten Shaker style configurations) found one new flaky human test and reproduced the jitter test in 3 of 20 runs. We also reran all 474 flagged tests about 350 times each on the 2 vCPU Linux runners with stress-ng running the whole time, 164,420 executions. No new flaky tests appeared. Only the two known Haiku tests flipped, and the 27 tests that always failed were all tests the one run filter had already rejected. With the October 4 run, every flagged test has now been run more than 1,300 times on the slower machine.

5 SCRIPTS, CODE, AND DATA

Everything is in the public GitHub repository https://github.com/shaunak1206/ece382v-flaky-llm-tests. The main pieces are below.

- `scripts/rerun.py`: reruns a suite N times in one mode (`both`, `tz`, `stress`, `stressng`) and records every test outcome from JUnit XML.
- `scripts/filter_oneshot.py`: the one run generate and filter step.
- `scripts/diagnose.py`: targeted reruns of each flaky test alone and in the suite under each change.
- `scripts/smells.py`, `run_smells.py`, `validate_scanner.py`: the static risk scanner and its validation.
- `scripts/select_subjects.py`: the subject selection funnel.
- `scripts/build_testpilot_context.py`: the TestPilot style prompt context.
- `scripts/mixed.py` and `.github/workflows/mixed.yml`: mixed suites on GitHub Actions.
- `scripts/extended.py` and `.github/workflows/extended.yml`: 1,000 reruns of flagged tests on Linux runners, with optional stress-ng.
- `scripts/coverage.py` and `analyze.py`: coverage and the tables in this report.
- Generated suites in `llm_tests/`, `gen_oneshot_sonnet/`, `gen_oneshot_haiku/`, `gen_tp_sonnet/`, `gen_plain_sonnet/`. Raw rerun results in `results/`, prompts in `prompts/`, the running write up in `REPORT.md`.

6 CHALLENGES

- **Flakiness is rare.** Under 0.2% of tests flip in any condition, so most comparisons need risk measures, failure rates, or much longer runs.
- **Generators do not always follow instructions.** Several one shot agents ran small shell commands (`mkdir`, `grep`, a syntax check) even though they were told not to run anything. None ran a test, so no suite got execution feedback, and every case is logged in `prompts/README.md`. One TestPilot style agent only read the first half of a long API file for more-itertools.

- **Rate limits.** The subject funnel first used the GitHub API and hit the hourly limit, so it now lists repository files with blobless git clones.
- **Two operating systems.** Coverage of filelock is low everywhere because the Windows module never runs on macOS or Linux, and timing behaves differently on the laptop and the runners. We report each machine separately.

7 NEXT STEPS (INTERMEDIATE STEPS 2, BY NOVEMBER 6)

(1) Add a second model family (a GPT class model) under the plain and TestPilot style prompts, since prompt style mattered more than model so far.
(2) Add four new subjects from the funnel pool (two per stratum) with the same steps, to show the funnel works for expansion.
(3) Run Shaker's configuration selection on the per run stress-ng records once enough flaky tests are collected, and repeat the stress-ng runs on the laptop.
(4) Run an iDFlakies style polluter and victim search on the mixed suites, starting from the schedule tests above.
(5) Try FlaPy [1] on our generated suites as an independent detector, and add an import level check to the scanner, since most of its misses have the nondeterminism inside the library.
(6) Start the mitigation early with the drafted prompt, comparing risk rate, flaky tests, coverage, and cost against a 3 and 5 run filter.

Division of work

So far Manas did the literature review and labeled the root cause of every flaky test, and Shaunak wrote the scripts, generated the suites, and ran the experiments and analysis. For the next milestone Manas will label the root causes of any new flaky tests and of the failures we trigger with stress-ng and delay injection, and write the related work section. Shaunak will add the second model family and the new subjects, run the Shaker configuration step and the mitigation experiments, and keep the analysis up to date.

REFERENCES

[1] Martin Gruber, Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. 2021. An Empirical Study of Flaky Tests in Python. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 148–158.
[2] Alex Gyori, Ben Lambeth, August Shi, Owolabi Legunsen, and Darko Marinov. 2016. NonDex: A Tool for Detecting and Debugging Wrong Assumptions on Java API Specifications. In *ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), Tool Demonstrations*.
[3] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 643–653.
[4] Shanto Rahman, Aaron Massey, Wing Lam, August Shi, and Jonathan Bell. 2024. Automatically Reproducing Timing-Dependent Flaky-Test Failures. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*.
[5] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105.
[6] Denini Silva, Leopoldo Teixeira, and Marcelo d'Amorim. 2020. Shake It! Detecting Flaky Tests Caused by Concurrency with Shaker. In *IEEE International Conference on Software Maintenance and Evolution (ICSME)*.